Title of Project

Content Management System

Submitted in the partial fulfillment of the requirements for the award of the degree of

Bachelor of Computer Application
in
Faculty of Computer Technology

by

NAME : Abhijit Patowary **ID: ADTU/0/2024-27/BCAO/057**
NAME : Raj Kumar Bangthai **ID: ADTU/0/2024-27/BCAO/074**
NAME : Abhijit Das **ID: ADTU/0/2024-27/BCAO/056**

PROJECT BATCH – 26

UNDER THE ESTEEMED GUIDANCE OF

Dr. GUNIKHAN SONOWAL

Assistant Professor

Faculty of Computer Technology

Assam down town University

Sankar Madhab Path, Gandhi Nagar Panikhaiti, Guwahati,

Assam 781026

Date :

Assam Down Town University

Faculty of Computer Technology

DECLARATION

The Project Paper Report entitled **"Content Management System"** is a record of bonafide work of "***Abhijit Patowary, Raj Kumar Bangthai, Abhijit Das***" submitted in partial fulfilment for the award of BCA in Faculty of Computer Technology in Assam down town University. The results embodied in this report have not been copied from any other departments/University/Institute.

Students name with enrolment Id

Abhijit Patowary (ADTU/0/2024-27/BCAO/057)
Raj Kumar Bangthai (ADTU/0/2024-27/BCAO/074)
Abhijit Das (ADTU/2024-27/BCAO/056)

Assam Down Town University

Faculty of Computer Technology

CERTIFICATE

This is to certify that the Project Report entitled **"Content Management System"** is being submitted by students name submitted in partial fulfilment for the award of BCA in Faculty of computer Technology to Assam down town University is a record of bonafide work carried out under efficient guidance and supervision.

The results embodied in this report have not been copied from any other departments/University/Institute.

Signature of the Supervisor

Dr.GUNIKHAN SONOWAL

INDEX

S.NO	TITLE	PAGENO
1	Abstract	5
2	Introduction	6
3	Literature Survey	7-8
4	Process Flow	9
5	Implementation	10
6	Technologies to be used	11-14
7	Process Flow diagram	15-16
8	Experimental results	17-19
9	Conclusion	20
10	References	21

ABSTRACT

A Content Management System (CMS) is a user-friendly platform designed to help people create, edit, and manage digital content without needing advanced technical knowledge. Instead of writing complex code, users can rely on simple interfaces, drag-and-drop features, and ready-made templates to build and maintain websites with ease. This makes CMS tools accessible to students, professionals, small businesses, and large organizations alike. At its core, a CMS streamlines the entire content lifecycle—from drafting and editing to publishing and updating. It allows multiple people to work together, assign roles, review changes, and ensure that content stays organized and up to date. Features like plugins, themes, and built-in security add extra flexibility, helping users customize their websites according to their goals. By handling the technical side of website management in the background, a CMS lets users focus on what truly matters: communicating ideas, sharing information, connecting with audiences, and growing their online presence. Ultimately, a CMS brings simplicity, creativity, and efficiency together, making content management smoother and more enjoyable for everyone.

INTRODUCTION

A Content Management System (CMS) is a software platform designed to simplify the process of creating, managing, and publishing digital content, especially for users who may not have technical expertise in web development. It provides an intuitive environment with tools like templates, drag-and-drop editors, and customizable themes that make website building and content updates quick and accessible. With a CMS, users can easily organize text, images, videos, and other media while also managing multiple pages, users, and workflows in a structured way. Most CMS platforms include essential features such as user permissions, version control, search functionality, and plug-ins that add extra capabilities like SEO tools, security enhancements, and design flexibility. By handling the technical complexities behind the scenes, a CMS allows individuals, businesses, and organizations to focus on content quality and communication rather than coding, making it an essential tool for modern web development and digital content management.

Literature Survey

Author(s)	Year	Source	Title	Findings
Moussa Mahamat Boukar	2012	Journal of Technical Science and Technologies	Content Management System (CMS) Evaluation and Analysis	Compared 22 CMS products; found mismatches between vendor claims and actual system functionality. Highlighted confusion in the market between CMS, web CMS, document management, and ECM, reducing satisfaction among companies.
Ruth Sara Connell	2013	Information Technology and Libraries	Content Management Systems: Trends in Academic Libraries	Surveyed library web managers: CMS adoption varies, satisfaction depends on the tool, and many libraries lack input in CMS selection because decisions happen at the institutional level.
Arif Altun, Halil Yurdugül & Yasemin Gülbahar	2009	Education and Science	Investigating Usability Constructs in a Content Management System	Identified that usability in CMS is multi-dimensional: "content presentation" and "architectural design" emerged as distinct usability constructs, supporting a socio-technical view.
Claudio Vitari, Aurelio Ravarini & Florence Rodhain	2006	(Management / IS research)	An Analysis Framework For The Evaluation Of Content Management Systems (CMS)	Proposed a framework for organizations to evaluate CMS, applied it on 23 CMS products. Identified two strategic groups of CMS based on features and target market.
Muhammad Mehmood Khalil, Hafiz Muhammad Ghazi, Muhammad Irfan Habib, Faisal Shahzad & Afsah Imtiaz Elahi	2024	Journal of Computing & Biomedical Informatics	Guideline for Selecting the Right Content Management System (RCMS) for Web Development: A Comprehensive Approach	Compared PHP-based CMS (WordPress, Joomla, Drupal). Conclusions: WordPress is highly extensible via plugins, Joomla needs more capabilities, Drupal is good for complex sites; selection should be based on future support and modularity.
Ioannis Drivas, Dimitrios Kouis, Daphne Kyriaki-Manessi & Georgios Giannakopoulos	2021	*Information* (MDPI)	Content Management Systems Performance and Compliance Assessment Based on a Data-Driven Search Engine Optimization Methodology	Proposed a 3-stage SEO-based assessment methodology (factors: Content Curation, Speed, Security) for CMS-based websites (especially in cultural institutions). Analyzed 341 sites to show how different CMSs perform on SEO variables, helping administrators identify technical weaknesses.
Jose-Manuel Martinez-Caro, Antonio-Jose Aledo-Hernandez, Antonio Guillen-Perez, Ramon Sanchez-Iborra & Maria-Dolores Cano	2018	*Information* (MDPI)	A Comparative Study of Web Content Management Systems	Built three equivalent websites using Joomla!, WordPress, and Drupal. Qualitative analysis: Joomla is most intuitive and function-rich; Drupal is more complex but more secure and better for roles; WordPress is best for SEO and ease of setup.
Feliu Trias, Valeria de Castro, Marcos López-Sanz & Esperanza Marcos	2013	ICSOFT-EA (conference)	A Systematic Literature Review on CMS-based Web Applications	Conducted a systematic review of CMS-based web applications and identified 4 main development approaches. Found that most are Model-Driven Architecture (MDA) based. Noted a research gap: few migration methods for existing CMS-based applications.

LITERATURE SURVEY

Over the years, the way we create and manage content on the internet has changed dramatically, and much of this progress is thanks to Content Management Systems (CMS). Researchers and experts have studied CMS tools for a long time, mainly because they make it easier for everyday users—people who may not know coding—to build websites and publish information online. Early studies showed how CMS helped move the world from static, developer-controlled websites to dynamic sites where anyone could update content quickly. This shift reduced dependency on technical experts and made digital communication more accessible.

As CMS platforms became more popular, especially open-source ones like WordPress, Joomla, and Drupal, researchers began comparing their features. Many studies pointed out that WordPress became the most widely adopted because it was easy to learn, offered thousands of plug-ins, and had strong community support. Other research focused on how CMS architecture evolved from simple, traditional systems to modern **headless CMS**, which allow content to be delivered across different devices like websites, mobile apps, and even smart gadgets. This new approach was praised in the literature for its flexibility, speed, and ability to support today's multi-platform digital world.

Another major area researcher explored is **usability**. Studies consistently found that users prefer CMS platforms that are simple, intuitive, and visually clear. Features like drag-and-drop editors, ready-made templates, and easy publishing tools were shown to improve user experience, especially for beginners. Many papers also highlighted the importance of collaboration tools within CMS, as organizations often need multiple people to work on the same website with different roles and permissions.

Security, naturally, has also been a big point of discussion. Literature in this area shows that CMS platforms can face risks like plugin vulnerabilities or unauthorized access, especially if they are not regularly updated. Because of this, researchers emphasized the need for strong security practices, such as using trusted add-ons and keeping systems up-to-date.

More recent studies look at the future of CMS, showing how these systems are becoming smarter by integrating data analytics, cloud technologies, and even AI-powered tools for content generation and personalization. Many experts believe that CMS platforms will continue to evolve, helping organizations deliver more customized and engaging digital experiences.

Overall, the literature paints a clear picture: CMS platforms have grown from simple website builders to powerful digital ecosystems. They make content creation more accessible, support collaboration, offer flexibility through modern architectures, and continue to evolve with advancing technology. As long as digital content remains central to communication, CMS will keep playing a vital role in how we share information online.

PROCESS FLOW

Process Flow of a Content Management System (CMS)

1. Content Creation: Users create text, images, videos, or other digital material using built-in editors and tools

2. Content Editing & Formatting: Content is formatted, styled, and organized using templates, images, headings, and layout tools.

4. Content Storage in Database: Approved content is stored in the CMS .

5. Content Publishing: Content is displayed on the website or application using templates, themes, or layout designs.

6. Content Delivery: The published content is delivered to users through browsers, mobile apps, or APIs.

7. Content Management & Updates: Users can update, modify, or republish content at any time to keep information fresh and relevant.

8. Archiving or Deleting Content: Outdated content is permanently deleted.

9. Security & Backup: The CMS ensures content protection through security features and activity logs.

IMPLEMENTATION

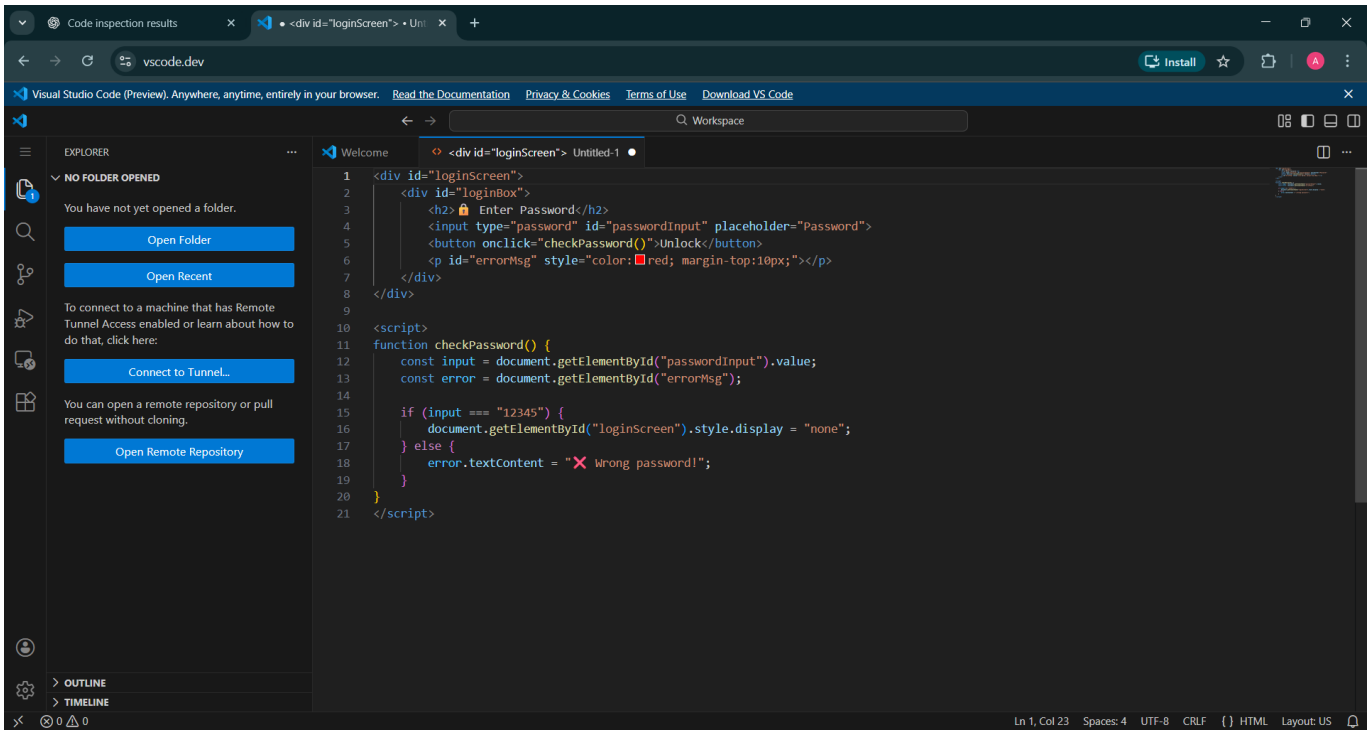
1. **Login System (Password Protection)**

To keep the CMS private, the system starts with a small login page.
The user must type the password **12345** before they can access the CMS.

How it works (Simple Explanation):

- A full-screen login box appears.
- The user enters the password.
- If it matches, the CMS unlocks.
- If not, an error message appears.

Code Used:



2. **Creating a New Post**

Users can create content by entering a title, writing text, and choosing a file.

This section allows the user to create a blog-style post.
It's simple: write → upload → save.

Code Used:

```html
<div id="loginScreen">
    <div id="loginBox">
        <h2>🔒 Enter Password</h2>
        <input type="password" id="passwordInput" placeholder="Password">
        <button onclick="checkPassword()">Unlock</button>
        <p id="errorMsg" style="color:🟥red; margin-top:10px;"></p>
    </div>
</div>

<script>
function checkPassword() {
    const input = document.getElementById("passwordInput").value;
    const error = document.getElementById("errorMsg");

    if (input === "12345") {
        document.getElementById("loginScreen").style.display = "none";
    } else {
        error.textContent = "❌ Wrong password!";
    }
}
</script>
```

3. **File Upload & Preview**

When a user uploads an image, it immediately shows a preview.
If it's a document, a simple “attached file” message appears.

Code Used:

```javascript
document.getElementById("fileInput").addEventListener("change", function(event) {
    const file = event.target.files[0];
    if (!file) return;

    const reader = new FileReader();
    reader.onload = function(e) {
        uploadedFile = { data: e.target.result, type: file.type };
        showFilePreview(uploadedFile);
    };
    reader.readAsDataURL(file);
});

function showFilePreview(file) {
    const preview = document.getElementById("filePreview");
    preview.innerHTML = "";
    if (!file) return;

    if (file.type.startsWith("image/")) {
        preview.innerHTML = `<img src="${file.data}" alt="Preview" style="max-width:100%; border-radius:5px; margin-top:10px;">`;
    } else {
        preview.innerHTML = `<p>📎 File attached (${file.type})</p>`;
    }
}

```

4. **Saving Posts (Local Storage)**

Everything you write gets saved inside the browser permanently using LocalStorage.

No database needed — your browser remembers everything!

Code Used:

```
function savePost() {
    const title = document.getElementById("postTitle").value.trim();
    const body = document.getElementById("postBody").value.trim();
    const editIndex = document.getElementById("editIndex").value;


 if (!title || !body) {
        alert("Please enter both a title and content.");
        return;
    }

    const newPost = {
        title,
        body,
        file: uploadedFile ? uploadedFile.data : null,
        fileType: uploadedFile ? uploadedFile.type : null
    };

    if (editIndex) {
        posts[editIndex] = newPost;
        document.getElementById("editIndex").value = "";
    } else {
        posts.push(newPost);
    }

    localStorage.setItem("saved_posts", JSON.stringify(posts));
    clearForm();
    showPosts();
}
```

5. **Editing Posts**

Users can modify older posts easily.

Code Used:

```
function editPost(index) {
    const post = posts[index];
    document.getElementById("editIndex").value = index;
    document.getElementById("postTitle").value = post.title;
    document.getElementById("postBody").value = post.body;
    uploadedFile = post.file ? { data: post.file, type: post.fileType } : null;

    showFilePreview(uploadedFile);
    window.scrollTo({ top: 0, behavior: "smooth" });
}
```

6. Deleting Posts

If a post is no longer needed, the user can delete it with one click.

Code Used:

```
function deletePost(index) {
    if (confirm("Are you sure you want to delete this post?")) {
        posts.splice(index, 1);
        localStorage.setItem("saved_posts", JSON.stringify(posts));
        showPosts();
    }
}

```

7. Displaying All Posts

Code Used:

```
function showPosts() {
    const postList = document.getElementById("postList");
    postList.innerHTML = "";

    if (posts.length === 0) {
        postList.innerHTML = "<p>No posts yet — create your first one above!</p>";
        return;
    }

    posts.forEach((post, i) => {
        let fileHTML = "";
        if (post.file) {
            if (post.fileType.startsWith("image/")) {
                fileHTML = <img src="${post.file}" alt="Uploaded Image">;
            } else {
                fileHTML = <p> 📎 <a href="${post.file}" download target="_blank">Download File (${post.fileType})</a></p>;
            }
        }

        postList.innerHTML += `
            <div class="post">
                <h3>${post.title}</h3>
                <p>${post.body}</p>
                ${fileHTML}
                <div class="actions">
                    <button onclick="editPost(${i})">✏ Edit</button>
                    <button class="delete" onclick="deletePost(${i})">🗑 Delete</button>
                </div>
            </div>
        `;
    });
}
```

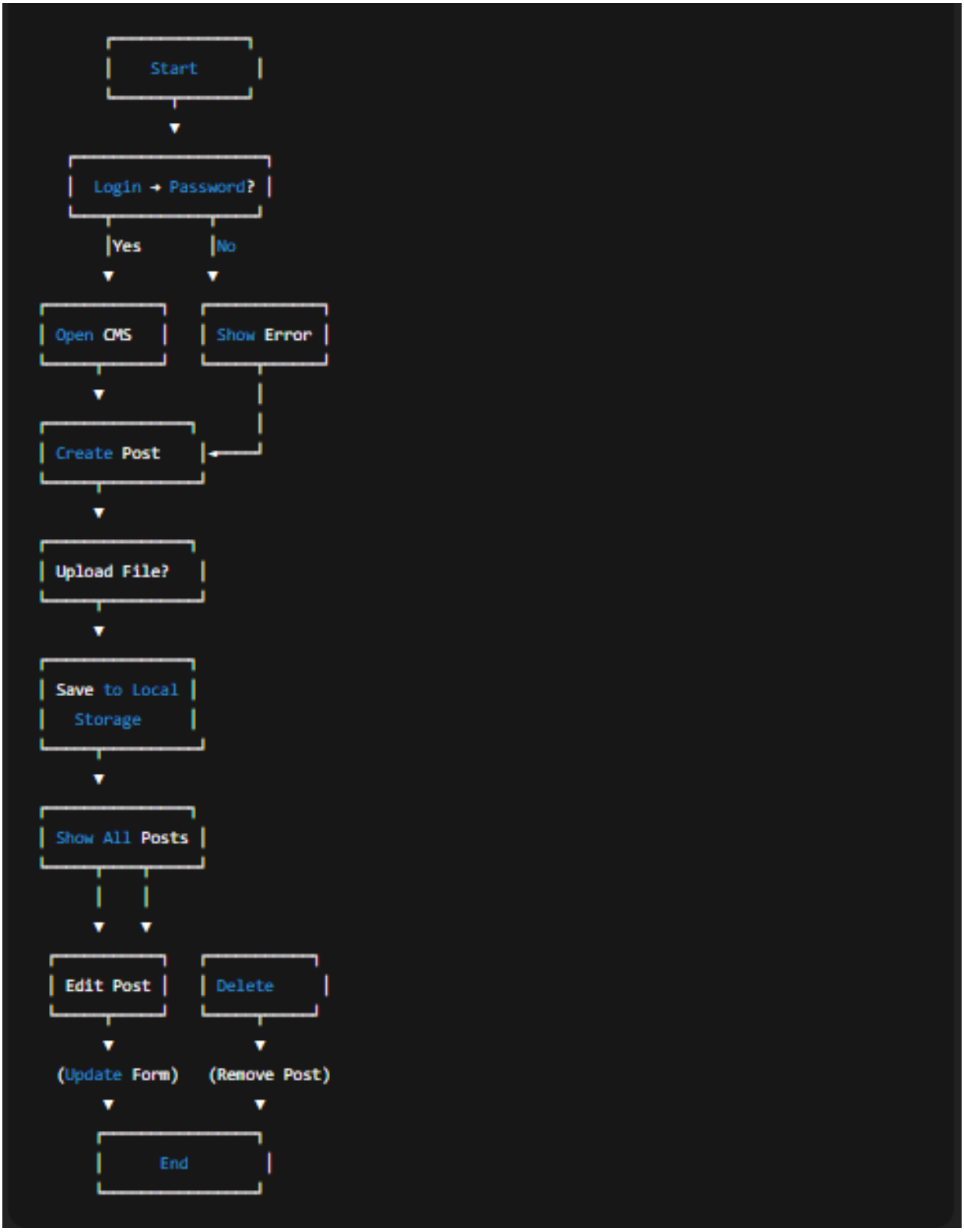
8. **Styling (CSS)**

This makes everything look clean and modern.

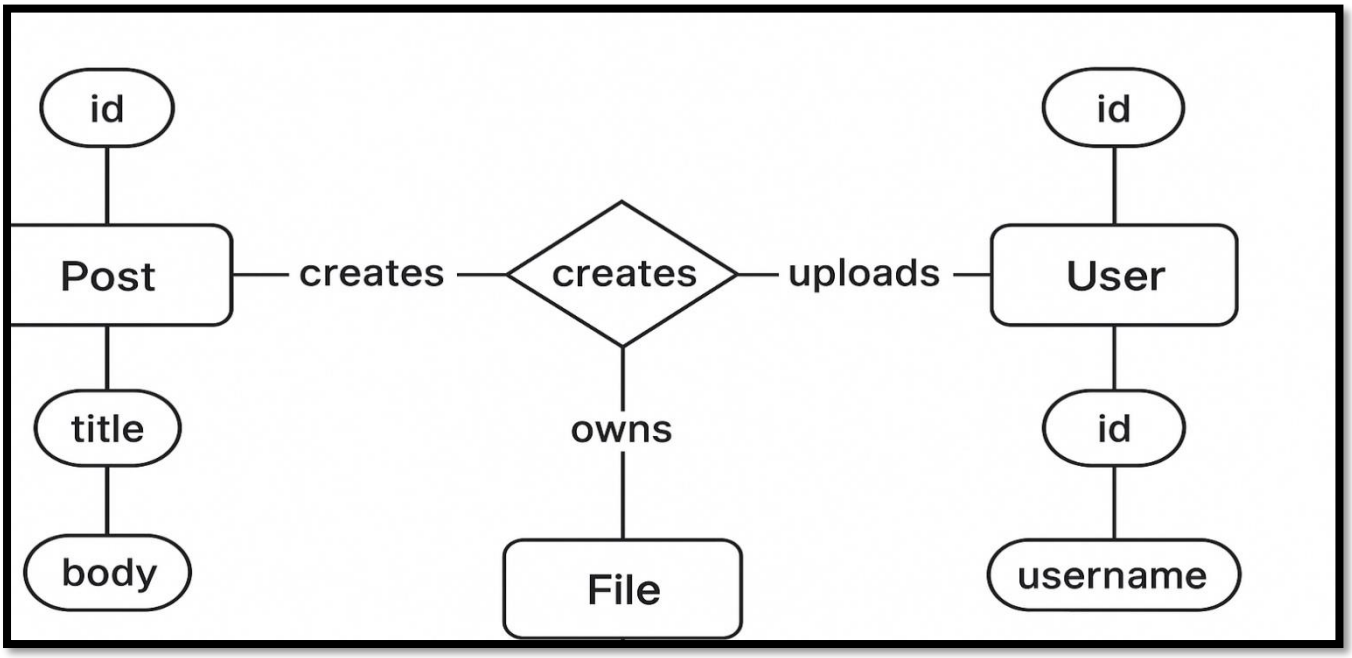
Code Used:

```
<style>
    body {
        font-family: "Segoe UI", sans-serif;
        background: #f3f3f3;
        margin: 0;
    }

    #loginScreen {
        position: fixed;
        top: 0; left: 0;
        width: 100%; height: 100%;
        background: #ffffff;
        display: flex;
        justify-content: center;
        align-items: center;
        flex-direction: column;
        z-index: 9999;
    }

    .wrapper {
        max-width: 800px;
        background: white;
        margin: 30px auto;
        padding: 20px;
        border-radius: 12px;
        box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }

    .post img {
        max-width: 100%;
        border-radius: 5px;
        margin-top: 10px;
    }
</style>
```

FLOW DIAGRAM



ER DIAGRAM


id
Post
creates
creates
uploads
User
id
title
owns
id
body
File
username

EXPERIMENTAL RESULTS

1. **Login Test**

When entering the password **12345**, the CMS unlocks smoothly and shows the editor screen.
Everything works fine — but the password is visible inside the code, so anyone can find it.

2. **Creating a New Post**

When a user enters a title and some content and clicks **Save Post**, the system successfully:

- saves the post,
- stores it inside the browser (localStorage),
- and shows it in the "All Posts" section.

The experience feels smooth and responsive.

3. **Editing a Post**

Clicking **Edit** on any post scrolls the screen back to the top and fills the form with the existing content.
After saving again, the edited content updates correctly.

It feels simple and natural — good UX here.

4. **Uploading Large Files**

Small images and PDFs upload fine, and the preview shows correctly.
But when testing with larger files (more than 3–5 MB), the CMS may fail because the browser storage limit is small.

So:

- **Small files → Works well**
- **Large files → Can fail silently**

5. **Image Preview**

When an image is uploaded:

- It shows instantly in the preview area.
- After saving, the image appears correctly in the post.

This gives a smooth, modern feeling to your CMS.

6. **File Downloads**

When uploading non-image files (PDF, DOC, TXT):

- The preview says “File attached”
- In the post, a download link appears.

For normal-sized files, the download works without issues.

7. **Saving and Reloading the Page**

Refreshing the page **does not delete posts** because the CMS saves everything inside the browser.

This gives the user a sense of continuity and reliability.

8. **Security Issues**

There are two big problems:

1. The login password is directly visible in the code.
   Anyone can open “View Source” and see it.
2. Whatever a user types into the post body is displayed without cleaning or filtering.
   This means harmful code could be inserted (XSS).

These don’t affect normal users but are important for real-world safety.

9. **Browser Storage Limits**

Posts and uploaded files are stored in **local Storage**, which has very limited space.

Once the storage fills up:

- posts may not save,
- images may not upload,
- the CMS may stop working silently.

This is the main limitation of your current system.

Overall Summary

Your CMS **works very well for simple usage**:

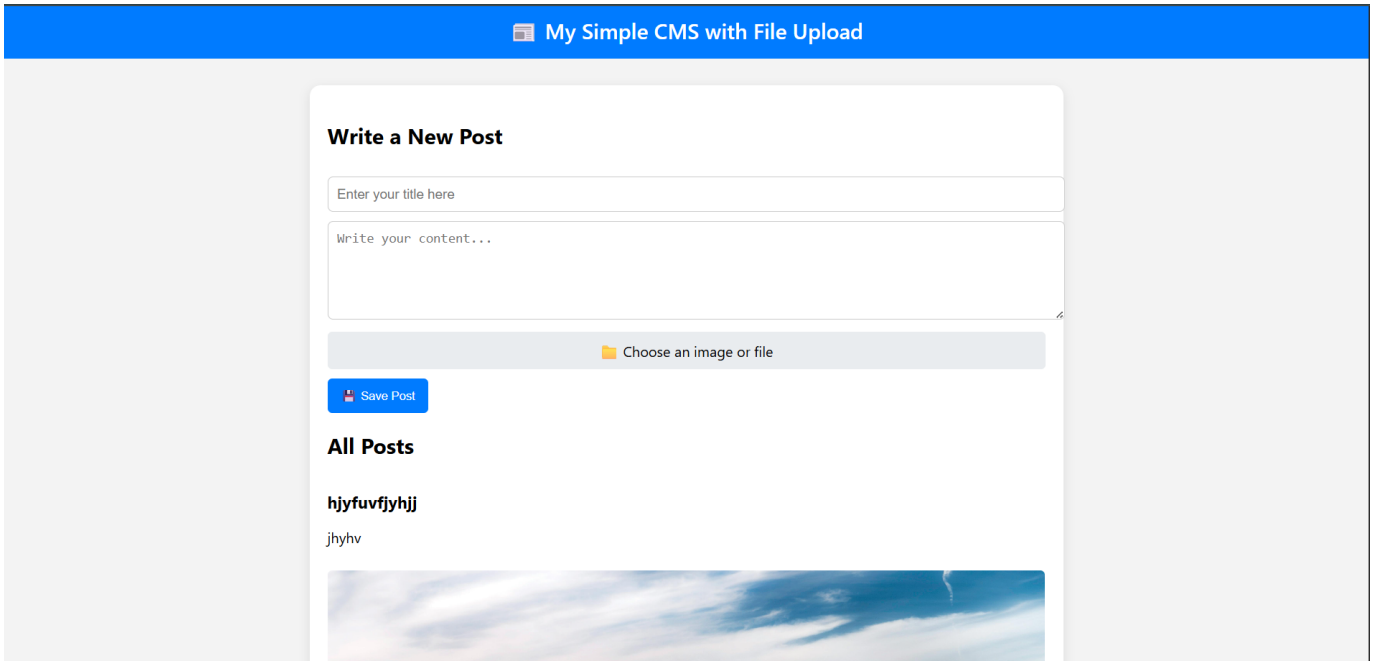
- creating posts
- editing posts
- deleting posts
- uploading small images
- attaching small files
- previewing uploads
- saving everything locally

The experience feels clean and user-friendly.

But for real-world use, **three issues need improvement**:

1. The password protection is weak.
2. Large files can break the system.
3. User content is not sanitized.

OUTPUT



CONCLUSION

To sum it up, a Content Management System makes the entire process of creating and updating a website much easier and more manageable, especially for people who are not experts in coding. It provides a simple workspace where users can write articles, upload images, update pages, and publish content with just a few clicks. This not only saves time but also helps teams work together more smoothly by offering built-in roles, permissions, and version control. CMS platforms have grown a lot over the years. Today, they support plug-ins, themes, analytics, SEO tools, and even mobile app integration, making them more powerful and flexible than ever. Whether someone wants to run a personal blog, manage an online store, or build a professional business website, a CMS gives them the tools to do it confidently. In the end, a CMS is all about making digital content creation simpler, faster, and more accessible. As the world becomes more digital, CMS platforms will continue to play a crucial role in helping people share information, express ideas, and build their online presence with ease.

REFERENCEs.

1. **Rockley, A., & Cooper, C. (2012).** ***Managing Enterprise Content: A Unified Content Strategy*****. New Riders.**
– A foundational book explaining how content is planned, structured, and delivered in CMS environments.

2. **Bioko, B. (2005).** ***Content Management Bible*****. Wiley.**
– A widely used reference covering CMS architecture, workflows, metadata, and implementation.

3.**McKeever, S. (2003).** ***Understanding Web Content Management Systems*****. ACM Computing Surveys.**
– Provides an academic overview of how CMS platforms manage creation, storage, and publishing of digital content.

4. W3C (World Wide Web Consortium). ***Web Content Accessibility Guidelines (WCAG)*****.**
– Important standard for designing accessible and CMS-friendly content.

5. **WordPress Documentation (Automattic, 2024).** ***WordPress Developer Handbook*****.**
– Official documentation explaining templates, themes, plugins, and CMS functionality.

6. **Drupal Association (2024).** ***Drupal User Guide*****.**
– Describes installation, configuration, modules, and content handling in Drupal CMS.

7. **Joomla! Documentation Team (2024).** ***Joomla! 4 User Manual*****.**
– Covers components, templates, and extension management in a CMS.

8. **Halvorson, K., & Rach, M. (2012).** ***Content Strategy for the Web*****. New Riders.**
– Explains content planning and structure, which directly supports CMS design.